Epics & Stories — Bahria Town Karachi Home-Services Marketplace

bahria-tenders - Epic Breakdown

Overview

This document provides the complete epic and story breakdown for the Bahria Town Karachi home-services marketplace, decomposing the requirements from the PRD and Architecture (no separate UX doc — Bubble supplies the UI) into implementable stories. Epics map to the architecture's validation-first build order (Phase 0 concierge → Phase 1 Foundation → Phase 2 demand loop → Phase 3 trust loop → Phase 4 deferred machinery).

Requirements Inventory

Functional Requirements

FR1: Provider sign-up — register with phone (OTP), name, CNIC number, selfie, one+ Trades, primary Precinct; profile created in Pending state. FR2: Identity verification gate — mark Provider Verified/Pending; selfie must visibly match CNIC + founder vetting; only Verified Providers can be awarded Jobs. FR3: Public provider profile (humanized accountability) — show reputation in two layers: glance (rating + job count + positive trust line) and detail (honest dispute breakdown, provider reply, Adab sub-scores); deliberate new-provider zero-state. FR4: Persistent reputation record — permanent, cumulative, non-resettable; CNIC-unique accounts block reputation reset. FR5: Neighbor social proof — anonymous/aggregate precinct-level usage count ("used by N homes in your precinct"); never names in v1. FR6: Post a job — Resident posts Trade + description + Precinct; Job enters Open, visible to Verified matching Providers. FR7: Provider job discovery — Verified Provider sees Open Jobs matching their Trade(s)/area; exact address shared only after award. FR8: Submit a bid — Verified Provider responds with a single price + optional note; one active Bid per Job. FR9: Compare bids — Resident views all Bids with price + reputation + distance + social proof; not default-sorted by cheapest. FR10: Award the job — Resident awards to one Provider (→ Awarded); Job closes to bids; contacts shared. FR11: Offline completion — job execution + payment happen offline (cash); no in-app job payment. FR12: Mark job completed — Resident marks an Awarded Job Completed to enable rating. FR13: Rate the result — Resident gives overall Rating (e.g. 1–5) + optional review; updates cumulative reputation. FR14: Adab score — Resident rates Respect, Cleanup, Punctuality; stored/displayed distinctly from overall Rating. FR15: Raise a dispute — Resident flags a Completed Job with a reason → Pending Review (not public); notifies founder + provider. FR16: Founder review & legitimacy judgment — founder records outcome (Legitimate–Remedied / Legitimate–Unresolved / Not Upheld); only Legitimate outcomes affect public reputation. FR17: Subscription gating with free trial — first 2 months free; after trial, active Subscription

required to submit new Bids. FR18: Subscription payment & status — record/track Subscription status + renewal; manual collection in v1. FR19: Provider availability signal — Available/Unavailable toggle with auto-expiry after inactivity + per-job accept/decline. FR20: Broadcast & provider notification — on posting, push + SMS alert to all Verified, Available matching Providers. FR21: Supply-aware expectation setting — show honest expectation (who's online; response-time only once data exists; plain "no one online" state). FR22: Empty-state capture, resident notification & concierge backstop — leave-and-notify when empty; founder alerted to manually recruit/assign on no-bid Jobs. FR23: Provider right to respond — Provider submits their side of a Dispute before any reputation impact. FR24: Pattern weighting — repeated Legitimate Disputes weigh more than a single one. FR25: Resident standing (anti-abuse) — repeated Not-Upheld disputes lower the Resident's own standing.

NonFunctional Requirements

NFR1: Reputation integrity — reputation is permanent/cumulative/non-resettable; all aggregate writes server-side (Bubble backend workflows) with reputation fields non-client-writable; raw components stored + admin recompute button; CNIC-unique accounts. NFR2: Identity & data privacy — CNIC number + selfie are admin+owner only, never public; private-file storage (incognito-403 test); delete CNIC/selfie image after verification (retain hash/last-4); deny-by-default privacy rules; split Provider (public) / Provider Private (sensitive) data types. NFR3: Platform & feasibility — built solo, non-technical, on Bubble (no-code); real-time intentionally degraded to periodic refresh + push/SMS; no GPS/maps (precinct-based); no 3D/face-matching in v1. NFR4: Cost & sustainability — running costs covered by Subscription revenue; avoid per-job platform cost (no job payment processing) in v1. NFR5: Operational — verification, dispute adjudication, and the concierge backstop are founder-operated manual processes in v1. NFR6: Localization & accessibility (Urdu/English, low-tech users) — Western Arabic numerals app-wide; currency reads "Rs 1,500"; status labels paired with fixed Urdu translations; icon + text (never icon alone); generous tap targets; reassuring empty states; calm "panic-moment" notification copy.

Additional Requirements

(Architecture-derived — these shape Epic 1/Foundation and cross-cutting setup)

- **Platform setup ("starter"):** create the Bubble app on the **Web + Mobile** plan; confirm the plan supports **API Workflows** (server-side reputation/money writes) and **point-in-time restore**. This is the first foundation story.
- **Data model:** Bubble data types — User (role), Provider (public), Provider Private (CNIC/selfie/raw verification), Resident Profile (precinct, standing), Job, Bid, Rating, Dispute, Notification; Trade & Precinct as Option Sets (slug + display); reputation stored as raw components (`rating_sum` , `rating_count` , dispute counts) + cached aggregate.
- **Per-role privacy rules** (Resident / Provider / Admin), deny-by-default; Provider Private admin+owner only.
- **Backend workflows (`BE -`):** Broadcast Job; Apply Rating to Reputation; Apply Dispute Outcome (terminal only); Recompute Reputation; Flag Expiring Trials (daily); Purge CNIC/Selfie (post-verification); Send SMS (provider alert); Check Phone/CNIC Uniqueness (signup).
- **External integrations:** OneSignal (native push); SMS gateway for provider alerts; payment gateway (Safepay/PayPro) deferred to post-v1 (subscription manual at launch).

- **App structure:** one Bubble app, three role-gated surfaces (Resident mobile, Provider mobile, Admin web); single role-switch mobile binary with `res_*` / `prov_*` page trees + Role as first-class field (so a later split stays cheap); Admin stays in-app.
- **Store readiness:** privacy policy, data-use disclosure, **in-app account-deletion flow** (Apple-mandated), native-feel nav to clear Apple 4.2; budget 2–4 review cycles; Apple ($99/yr) + Google ($25) dev accounts.
- **Conventions:** maintain the one-page CONVENTIONS sheet (Glossary→Data Type map, prefix legend, locked enums, formats) as source of truth; environment discipline (never test on Live).
- **Build order:** Phase 0 (Wizard-of-Oz concierge, no real build) → Phase 1 Foundation → Phase 2 demand loop (early) + thin provider feed → Phase 3 trust loop → Phase 4 deferred heavy machinery (full admin verification queue, automated uniqueness/CNIC-purge, subscription automation).

UX Design Requirements

No standalone UX specification was produced (deliberate — Bubble supplies the UI for a no-code v1). UX-critical requirements are captured as NFR6 (localization/accessibility) and within FR3 (humanized provider card) and FR21/FR22 (honest expectation-setting + reassuring empty states), and should be honored at story level.

FR Coverage Map

- FR1 → Epic 1 (provider sign-up)
- FR2 → Epic 1 (verification gate)
- FR3 → Epic 3 (humanized provider card / profile display)
- FR4 → Epic 3 (persistent reputation; CNIC-uniqueness enforced in Epic 1)
- FR5 → Epic 3 (neighbor social proof)
- FR6 → Epic 2 (post a job)
- FR7 → Epic 2 (provider job discovery)
- FR8 → Epic 2 (submit a bid)
- FR9 → Epic 2 (compare bids)
- FR10 → Epic 2 (award the job)
- FR11 → Epic 2 (offline completion)
- FR12 → Epic 2 (mark job completed)
- FR13 → Epic 3 (rate the result)
- FR14 → Epic 3 (Adab score)
- FR15 → Epic 4 (raise a dispute)
- FR16 → Epic 4 (founder review & legitimacy judgment)
- FR17 → Epic 2 (bid-eligibility gate-check) + Epic 5 (billing)
- FR18 → Epic 5 (subscription payment & status)
- FR19 → Epic 2 (availability signal)
- FR20 → Epic 2 (broadcast & push+SMS notification)
- FR21 → Epic 2 (supply-aware expectation setting)
- FR22 → Epic 2 (empty-state capture + concierge backstop)
- FR23 → Epic 4 (provider right to respond)

- FR24 → Epic 4 (pattern weighting)
- FR25 → Epic 4 (resident standing)

Epic List

Milestone 0 — Concierge Validation (Phase 0, NO software build) 🧪

Before any build, the founder runs the Wizard-of-Oz loop with ~15–20 residents (manual intake → relay quotes → capture rating/Adab by hand) to validate that residents post & transact and that recourse matters. **A go/no-go gate above the epic list — not a software epic.**

Epic 1: Foundation & Trusted Provider Onboarding

A founder can stand up the Bubble app and onboard + verify the seeded providers; a verified provider profile exists with privacy locked down. **Scope is promoted to the full core-schema skeleton** to avoid live-data migrations on Bubble: define ALL core data types and their key fields now (even if later epics populate them) — `User` (role), `Provider` (incl. `avg_rating`, `rating_count`, `adab_score`, `subscription_status`, `subscription_expiry`), `Provider Private` (CNIC/selfie), `Resident Profile` (precinct, standing), `Job` (with the FULL status option-set: `posted→bidding→awarded→connected→completed→disputed→closed` and a `Dispute` link field), `Bid`, `Rating`, `Dispute`, `Notification`; `Trade` / `Precinct` Option Sets; **restrictive-first per-role privacy rules** (reputation fields non-client-writable from day one); and the **CNIC-uniqueness gate as a backend workflow** (not a client search). **FRs covered:** FR1, FR2 *(+ architecture Foundation/"starter": Bubble app + Web+Mobile plan, conventions sheet, NFR1/NFR2 scaffolding)*

Epic 2: The Job Loop — Post → Bid → Award → Connect

A resident posts a job; available, verified providers are alerted (push + SMS) and bid; the resident compares on reputation and awards; the two connect offline — with honest supply-aware expectation-setting, an empty-state/concierge backstop, and a light subscription gate-check (everyone is on `trial` at launch). **Full bid/award auction is in v1 from the start.** Note: the server-side **broadcast (FR19–22)** is its own backend/integration story cluster (scheduled API workflow + SMS connector + push plugin), distinct from the client-side loop. **FRs covered:** FR6, FR7, FR8, FR9, FR10, FR11, FR12, FR19, FR20, FR21, FR22, FR17 *(gate-check only)*

Epic 3: Reputation & Trust — Rating, Adab, Profile, Social Proof

After a job, residents rate the result and manners; provider reputation accumulates permanently (server-side writes into the Epic 1 fields) and surfaces humanely on the hire-time card, with anonymous neighbor social proof. **FRs covered:** FR3, FR4, FR5, FR13, FR14

Epic 4: Accountability & Recourse — Disputes

Residents have real recourse; a fair dispute system (provider right-to-respond, founder adjudication, pattern weighting) protects both sides, with anti-abuse resident standing. *In early v1, recourse can stay founder-operated until volume justifies the full software.* **FRs covered:** FR15, FR16, FR23, FR24, FR25

Epic 5: Provider Subscription & Sustainability (last; deferrable past initial launch)

Providers run a 2-month free trial then subscribe to keep bidding; the founder tracks status/expiry so the platform pays for itself. **The bid gate-check lives in Epic 2; this epic adds the billing + expiry-flag machinery** — which can launch ~2 months after go-live, since the trial covers everyone initially.
FRs covered: FR17, FR18

Epic 1: Foundation & Trusted Provider Onboarding

Stand up the Bubble app and onboard + verify the seeded providers, with the full data-model skeleton and privacy locked down so later epics never migrate live data.

Story 1.1: Create the Bubble app & confirm plan capabilities

As the **founder/builder**, I want the Bubble Web+Mobile app created and its plan capabilities confirmed, So that I have a foundation that supports the server-side and notification work later epics need.

Acceptance Criteria:

Given a new Bubble project **When** the app is created on the Web + Mobile plan **Then** API Workflows (server-side) are available **And** point-in-time restore is available on the plan **And** a `Role` Option Set (Resident/Provider/Admin) and the `res_*` / `prov_*` / `admin_*` page-prefix convention are established **And** a one-page CONVENTIONS sheet (Glossary→Data Type map, prefix legend, enums, formats) is started.

Story 1.2: Define the full core data-model skeleton

As the **founder/builder**, I want all core data types, key fields, and Option Sets defined up front, So that adding ratings/disputes/subscription later never forces a live-data migration.

Acceptance Criteria:

Given the Bubble database **When** the schema is created **Then** these data types exist with their key fields: `User` (role), `Provider` (trades, precinct, availability, `avg_rating`, `rating_count`, `adab_respect`, `adab_cleanup`, `adab_punctuality`, `subscription_status`, `subscription_expiry`), `Provider Private` (cnic, selfie, verification raw), `Resident Profile` (precinct, standing), `Job` (trade, description, precinct, status, dispute link), `Bid` (price, note, status), `Rating` (overall, review, adab fields), `Dispute` (reason, status, provider_response, admin_notes), `Notification` (provider, job, sent) **And** `Job.status` is an Option Set enumerating the FULL lifecycle (`posted` / `bidding` / `awarded` / `connected` / `completed` / `disputed` / `closed`) **And** `Trade` and `Precinct` Option Sets use ASCII slug internal names + Display text.

Story 1.3: Lock restrictive-first privacy rules

As the **founder**, I want privacy rules set deny-by-default with sensitive and reputation fields locked from day one, So that no CNIC leaks and reputation can never be written client-side.

Acceptance Criteria:

Given the core data types exist **When** privacy rules are configured **Then** every type defaults to no fields visible, exposed explicitly per role **And** `Provider Private` is visible to owner + admin only **And** the `cnic` / `selfie` file fields are private files (an incognito GET of the file URL returns 403) **And** `Provider` reputation fields (`avg_rating` , `rating_count` , `adab_*`) are NOT modifiable by the logged-in user (only API/admin context).

Story 1.4: Provider sign-up (FR1)

As a **prospective provider**, I want to register quickly with my identity and trades, So that I can start receiving jobs once verified.

Acceptance Criteria:

Given the Provider sign-up screen **When** a provider submits phone (OTP-verified), name, CNIC number, a selfie, one+ Trades, and primary Precinct **Then** a `Provider` (+ linked `Provider Private`) is created in `Pending` state **And** sign-up cannot complete if phone OTP, CNIC, selfie, or at least one Trade is missing **And** the CNIC/selfie are stored as private files.

Story 1.5: CNIC/phone uniqueness gate (NFR1)

As the **platform**, I want duplicate CNIC/phone registrations blocked server-side, So that a provider cannot abandon a bad reputation and start fresh.

Acceptance Criteria:

Given a provider submits sign-up **When** a backend (server-side) workflow checks the CNIC and phone against existing records **Then** a duplicate CNIC or phone is rejected before a new `Provider` is created **And** the check runs server-side (not a client-side search that could expose CNIC).

Story 1.6: Admin verification & post-verify purge (FR2)

As the **founder/admin**, I want to review and verify pending providers and then purge their raw ID images, So that only vetted providers win jobs and sensitive data isn't retained.

Acceptance Criteria:

Given a `Pending` provider in the admin review screen **When** the admin confirms the selfie visibly matches the CNIC and approves **Then** the provider is marked `Verified` **And** only `Verified` providers can have a Bid accepted **And** a `Pending` provider may browse but not win jobs **And** after verification, a workflow deletes the CNIC/selfie image (retaining only `verified=true` + a hash/last-4).

Epic 2: The Job Loop — Post → Bid → Award → Connect

A resident posts a job; verified, available providers are alerted and bid; the resident compares on reputation and awards; the two connect offline.

Story 2.1: Post a job (FR6)

As a **resident**, I want to post a small job for a trade in my precinct, So that nearby providers can respond.

Acceptance Criteria:

Given the Post-a-Job screen **When** the resident selects a Trade, enters a description, and confirms their Precinct **Then** a `Job` is created in `posted` / `Open` state visible to Verified providers in that Trade **And** the Job cannot be posted without a Trade and a Precinct.

Story 2.2: Provider availability signal (FR19)

As a **provider**, I want to set whether I'm currently available, So that I'm only alerted for jobs when I can take them, and residents see honest supply.

Acceptance Criteria:

Given a verified provider **When** they toggle Available/Unavailable **Then** the state is stored **And** availability auto-expires to Unavailable after N days of inactivity **And** only Available providers feed the "online now" signal and real-time alerts.

Story 2.3: Provider job discovery (FR7)

As a **provider**, I want to see open jobs matching my trades and area, So that I can choose which to bid on.

Acceptance Criteria:

Given a verified provider **When** they open the job feed **Then** they see only Open Jobs in Trades they offer **And** each Job shows precinct/distance but NOT the exact address/contact until the Job is awarded to them.

Story 2.4: Submit a bid with subscription gate-check (FR8, FR17 gate)

As a **provider**, I want to respond to a job with a price, So that I can win the work.

Acceptance Criteria:

Given an Open Job and a verified provider **When** they submit a single price + optional note **Then** at most one active Bid per Job exists (editable until award) **And** the bid is accepted only if the provider's `subscription_status` is `trial` or `active` (at launch everyone is `trial`) **And** bids are single-price (no tiers).

Story 2.5: Broadcast & provider notifications — backend (FR20)

As the **platform**, I want to alert all matching available providers when a job is posted, So that providers respond quickly even with push unreliability.

Acceptance Criteria:

Given a newly posted Job **When** a scheduled/backend API workflow runs **Then** all Verified + Available providers matching the Trade/area receive a **push (OneSignal) and SMS** alert with trade + area **And** one `Notification` row is written per provider with a `sent` flag (idempotent, retryable) **And** the broadcast runs server-side and does not block the resident's UI.

Story 2.6: Supply-aware expectation setting (FR21)

As a **resident**, I want an honest sense of whether providers will respond, So that I'm not left staring at silence.

Acceptance Criteria:

Given the resident is posting/awaiting bids **When** the screen renders **Then** with ≥1 matching Available provider it shows the online count **And** a response-time indication appears only once enough historical bid data exists (no fabricated ETA at launch) **And** with 0 Available providers it plainly says so (no real-time promise implied).

Story 2.7: Empty-state capture & concierge backstop (FR22)

As a **resident**, I want my request kept alive when no one responds, So that a thin early marketplace still resolves my job.

Acceptance Criteria:

Given a Job with no Available providers or no bids in the window **When** the empty state is shown **Then** the resident can leave the request and is notified when the first/each Bid arrives **And** the founder/admin is alerted to manually recruit/assign a provider **And** the Job stays Open (no dead-end).

Story 2.8: Compare bids & award (FR9, FR10)

As a **resident**, I want to compare responses and choose one, So that I hire on reputation, not just price.

Acceptance Criteria:

Given a Job with one or more Bids **When** the resident views the bids **Then** each Bid shows price + the provider's rating + distance + social proof, not default-sorted by cheapest **And** awarding moves the Job to `Awarded`, closes it to further bids, and shares each party's contact details.

Story 2.9: Offline completion & mark complete (FR11, FR12)

As a **resident**, I want to mark a job done after the provider finishes, So that I can then rate it.

Acceptance Criteria:

Given an Awarded Job **When** the work is done offline (paid in cash) **Then** there is no in-app payment/charge for the Job **And** the resident can mark the Job `Completed`, which enables rating.

Epic 3: Reputation & Trust — Rating, Adab, Profile, Social Proof

After a job, residents rate the result and manners; reputation accumulates permanently and surfaces humanely.

Story 3.1: Rate the result (FR13)

As a **resident**, I want to score a completed job and leave a review, So that good work is rewarded and bad work has consequences.

Acceptance Criteria:

Given a Completed Job **When** the resident submits an overall Rating (1–5) and optional review **Then** a `Rating` record is created **And** it triggers a server-side reputation update (Story 3.3) **And** only a Completed Job can be rated.

Story 3.2: Adab score (FR14)

As a **resident**, I want to rate respect, cleanup, and punctuality, So that manners count toward a provider's standing.

Acceptance Criteria:

Given a Completed Job being rated **When** the resident taps the three Adab dimensions (Respect, Cleanup, Punctuality) **Then** the Adab values are stored on the `Rating` **And** displayed distinctly from the overall Rating (lightweight, a few taps).

Story 3.3: Reputation aggregation & recompute (FR4, NFR1)

As the **platform**, I want reputation computed server-side from raw components with a recompute fallback, So that it stays permanent, consistent, and rebuildable.

Acceptance Criteria:

Given a new `Rating` (or terminal Dispute outcome) **When** a backend workflow runs **Then** it increments raw components (`rating_sum` / `rating_count` , Adab sums) and updates the cached `avg_rating` / `adab_*` on `Provider` **And** an admin "Recompute reputation from source" button rebuilds these from all `Rating` records **And** reputation cannot be reset/deleted by any in-app provider action.

Story 3.4: Humanized provider card / profile (FR3)

As a **resident**, I want to see a provider's track record clearly but humanely, So that I can judge trust without being misled or scaring off good providers.

Acceptance Criteria:

Given a provider with history **When** their card/profile renders **Then** the glance layer shows rating + job count + a positively-framed trust line ("83 jobs · 81 went smoothly") — never a raw unresolved-dispute count **And** a one-tap detail layer shows the honest dispute breakdown (unresolved = "we're looking into this") with the provider's reply **And** Adab sub-scores live in the detail layer **And** a new provider shows a deliberate "New · verified ID" zero-state.

Story 3.5: Neighbor social proof (FR5)

As a **resident**, I want to see how many neighbors used a provider, So that local trust informs my choice.

Acceptance Criteria:

Given a provider viewed by a resident **When** the card renders **Then** it shows "used by N homes in [resident's Precinct]" as an anonymous count (never names) **And** when N is 0 the signal is hidden/neutral (no fabricated numbers).

Epic 4: Accountability & Recourse — Disputes

A fair dispute system gives residents recourse while protecting providers from bad-faith flags.

Story 4.1: Raise a dispute (FR15)

As a **resident**, I want to flag a bad job, So that I'm not left helpless after poor work.

Acceptance Criteria:

Given a Completed Job the resident hired **When** they flag it with a short reason **Then** a `Dispute` is created in `pending_review` (not shown publicly) **And** the founder and the provider are notified.

Story 4.2: Provider right to respond (FR23)

As a **provider**, I want to give my side before any reputation impact, So that I'm not branded unfairly.

Acceptance Criteria:

Given a `pending_review` Dispute **When** the provider submits their response within the window **Then** the response is stored on the `Dispute` **And** no reputation change is applied while the Dispute is pending.

Story 4.3: Founder adjudication & reputation impact (FR16)

As the **founder**, I want to judge a dispute after hearing both sides, So that only legitimate problems affect a provider's standing.

Acceptance Criteria:

Given a `pending_review` Dispute with both sides **When** the admin records an outcome **Then** the outcome is one of `Legitimate-Remedied` / `Legitimate-Unresolved` / `Not Upheld` **And** only Legitimate outcomes affect public reputation (via Story 3.3) **And** a `Not Upheld` dispute never stains the provider and is not shown publicly.

Story 4.4: Pattern weighting (FR24)

As the **platform**, I want repeated legitimate disputes to weigh more than a single one, So that a one-off is forgiven but a pattern damages standing.

Acceptance Criteria:

Given a provider's legitimate-dispute history **When** reputation/standing is computed **Then** a single Legitimate dispute is low-weight (he-said/she-said) **And** a pattern materially lowers standing (threshold build-time tunable).

Story 4.5: Resident standing anti-abuse (FR25)

As the **platform**, I want bad-faith flagging to cost the resident, So that disputes aren't weaponized against good providers.

Acceptance Criteria:

Given a resident's dispute history **When** disputes are repeatedly judged `Not Upheld` **Then** the resident's internal standing is lowered **And** v1 may start by simply tracking it (effect build-time tunable).

Epic 5: Provider Subscription & Sustainability

Track trials and subscriptions so the platform pays for itself; deferrable until ~2 months post-launch.

Story 5.1: Subscription status tracking (FR18)

As the **founder**, I want to see and manage each provider's subscription state, So that I can collect fees and keep access correct.

Acceptance Criteria:

Given providers with `subscription_status` and `subscription_expiry` (from Epic 1) **When** the admin opens the subscription dashboard **Then** each provider's status (`trial` / `active` / `expired`) and expiry are visible **And** the admin can record a manual payment (bank transfer/JazzCash/Easypaisa) that sets status to `active` with a new expiry.

Story 5.2: Trial expiry flagging & gate enforcement (FR17 billing)

As the **platform**, I want trials and subscriptions to expire correctly, So that bidding access reflects who has paid after the free period.

Acceptance Criteria:

Given a daily backend workflow **When** it runs **Then** providers whose `trial` / `active` period is ending/ended are flagged for the admin **And** a provider whose status is `expired` cannot submit new Bids (Epic 2 gate-check) but keeps their profile and reputation **And** the 2-month trial starts at verification.